



PROJECT FIELD GUIDE

AuthShield 360

Project field guide and jury preparation

A practical overview of the school portal, its sign-in protections, supporting services, project structure and demonstration workflow.

SIGN-IN SEQUENCE

01

Password



02

SMS OTP



03

Email OTP

PROJECT TEAM

MSG Security Squad

TEAM LEAD

Mustafa Ahmed

TEAM MEMBERS

Munniba, Aliyan, Mehak, Aisha, Rabia

MENTOR

Sir Anas

COMPETITION

TechViz, held by Aptech

LIVE PORTAL

authshield360.vercel.app

SOURCE REPOSITORY

github.com/mustafaahmed-sec/AuthShield360

Private team study guide | Prepared 1 October 2026

For project understanding and authorized demonstration.

CONTENTS**Navigate the guide**

This guide explains the system from a first-time learner's point of view. Read the sign-in story first, then use the security and SRS sections to prepare for questions.

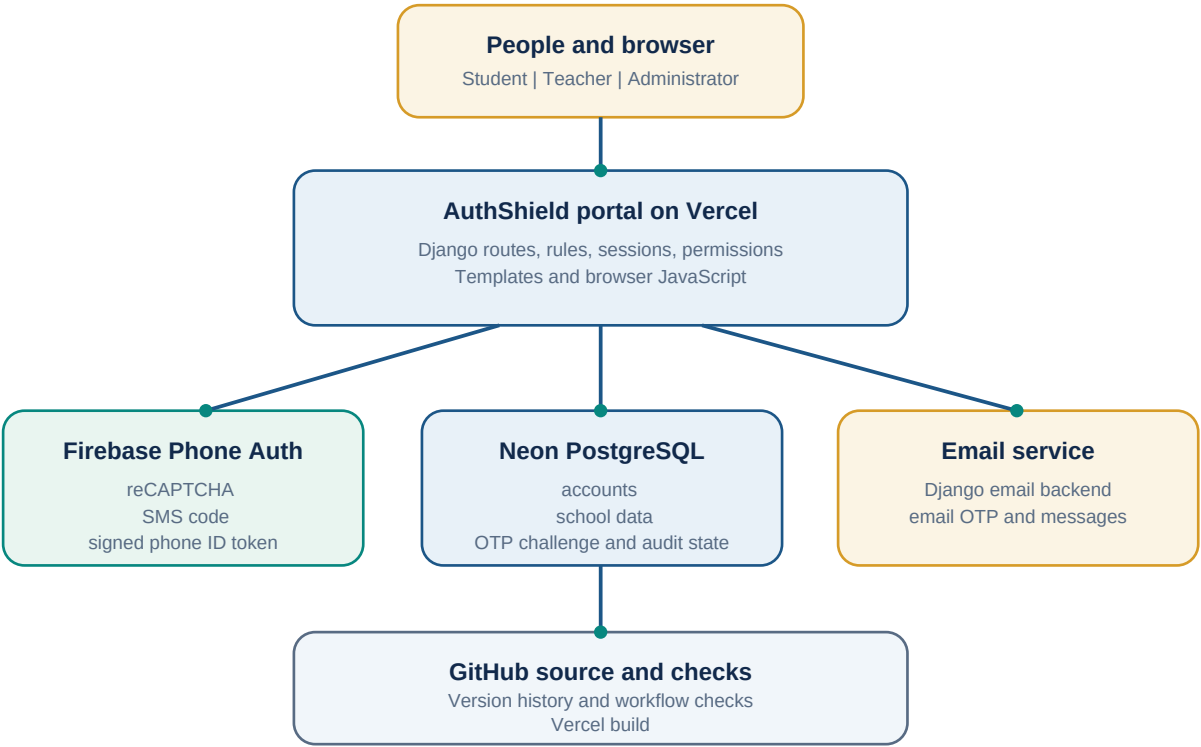
1. The project in one minute	3
2. Why AuthShield 360 exists	4
3. Scope and how to describe it	5
4. Application architecture	6
5. The technology, in plain language	7
6. Roles, permissions and account approval	8
7. Sign-in: SMS first, then email	9
8. What each sign-in step proves	10
9. Registration, reset and recovery	11
10. OTP limits and lockouts	12
11. MFA settings and Keycloak	13
12. Data, privacy and audit trail	14
13. Security controls and realistic limits	15
14. SRS traceability: implementation versus evidence	16
15. Test results and what they mean	17
16. Performance measurements: read the caveat	18
17. Source map and project handoff	19
18. Deployment and safe operation	20
19. Safe demonstration plan	21
20. Troubleshooting guide	22
21. Jury questions and concise answers	23
22. Roadmap: the next professional improvements	24
23. Glossary for a new team member	25
24. Sources and reference links	26

1. The project in one minute

AuthShield 360 is an educational school portal used to demonstrate identity security. It has Student, Teacher and Administrator areas. The main security idea is simple: a password alone is not enough. In the integrated sign-in flow, a person enters a password, verifies an SMS code through Firebase, then verifies an email code before Django grants a role-based portal session.

Scope of this document

This guide summarizes the project design and implementation for team learning. Use the SRS and evidence records for formal requirement review. Items marked pending still need verification before they are presented as complete.



Current implementation and evidence status

The portal source implements the SMS-first, email-second sequence and the production deployment was marked READY on 1 October 2026. The live /login/ page returned HTTP 200 and displayed the SMS-first instruction. The team previously reported receiving OTPs. No live sign-in or real SMS send was performed for this guide. The automated Django suite passed 136 tests using isolated test settings; that is not proof of carrier delivery or a penetration test.

Known item	Current value / careful interpretation
Event and team	TechViz, held by Aptech MSG Security Squad
Team	Mustafa Ahmed (lead); Munniba, Aliyan, Mehak, Aisha and Rabia; mentor Sir Anas
Public portal	https://authshield360.vercel.app
Source repository	https://github.com/mustafaahmed-sec/AuthShield360
Firebase credit	Team-reported USD 300 Firebase credits; not independently verified in this review.

2. Why AuthShield 360 exists

A school portal holds information people should not all see or change. A stolen password can expose an account when password-only sign-in is used. This project makes the authentication decision visible: it records a baseline concept, adds verification steps, enforces user roles, and keeps a record of security-relevant activity.

The system is a demonstration environment using fictional school records. The intended audience is students, teachers, administrators, mentors and evaluators learning how authentication, authorization and monitoring fit together.

Project objectives

- Show how a password and two additional verification channels work together.
- Separate Student, Teacher and Administrator capabilities.
- Require Administrator approval before a Student or Teacher request becomes an active portal account.
- Demonstrate code expiry, send limits, failed-login controls, audit events and recovery paths.
- Compare what the source implements with what the SRS requires teams to demonstrate and document.

What is outside the claim

This is not an unhackable product, a certification, a formal penetration-test result or proof that every carrier and country delivers SMS. The project does not authorize testing real school accounts or public third-party services. A successful application build is one part of readiness; SRS screenshots, controlled security tests and a demonstration video are separate evidence.

3. Scope and how to describe it

In the project scope	Meaning for a beginner
Identity	Which account is signing in? The portal identifies a user by email and checks their password.
Authentication	Is the person allowed to sign in? Password, SMS and email checks answer this in sequence.
Authorization	What may the signed-in person do? Django checks role and record relationships on each protected view.
Account lifecycle	Student and Teacher signup creates a request; an Administrator reviews and approves it.
School workflows	The portal models coursework, assignments, results, attendance, requests and administrative review.
Security monitoring	Audit events summarize relevant sign-in and account actions without storing readable codes or passwords.

A useful jury sentence

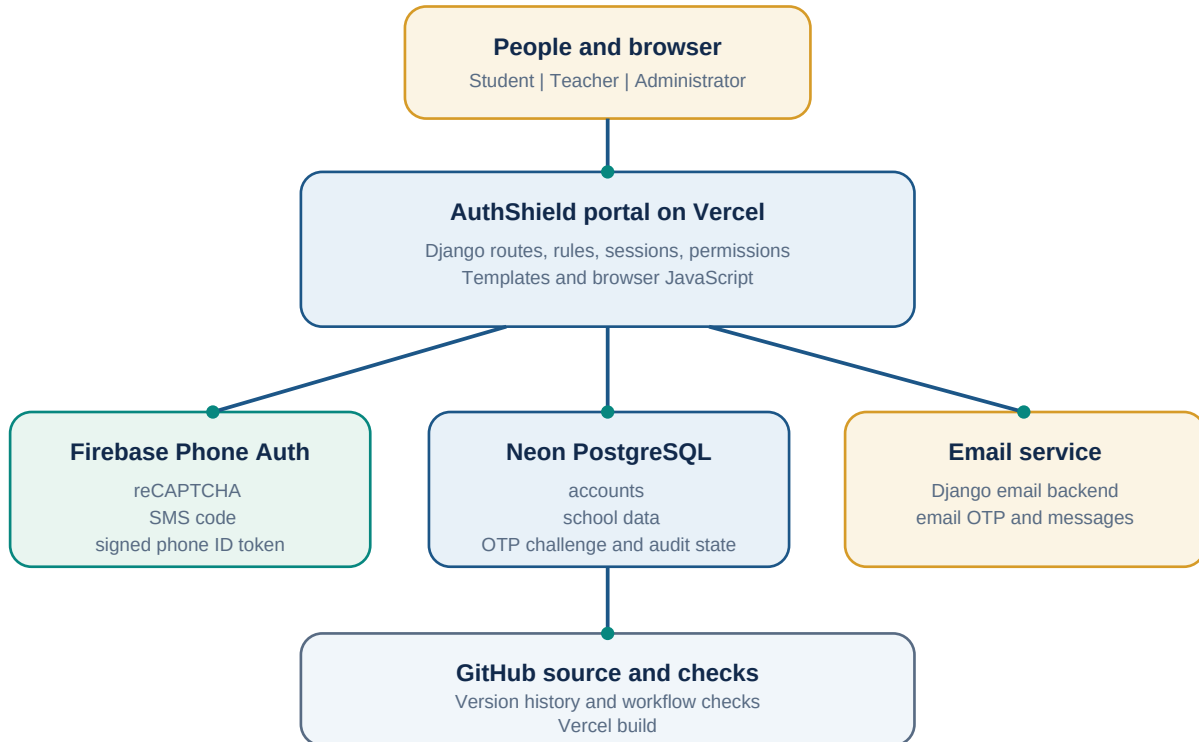
Explain the purpose

We built a fictional school portal to show how layered sign-in and role checks reduce the risk of a stolen password. SMS is verified first, email is verified second, and the application creates a session only after the required checks succeed.

Avoid saying that adding OTP makes an account impossible to compromise. MFA raises the effort needed for an attacker, but SIM swapping, mailbox compromise, phishing, stolen sessions, unsafe deployment settings and application defects still matter.

4. Application architecture

Think of the portal as a group of services with different jobs. The browser displays forms and talks to Django. Django is the decision-maker for the application: it checks passwords, permissions, approval state, challenge expiry and session creation. Firebase verifies the phone step. Neon persists the portal data. The email backend delivers a separate email code.



The arrows show service calls, not trust. A browser can be modified by a user, so Django verifies the signed Firebase ID token on the server. A browser message saying SMS succeeded is not sufficient by itself.

Request path	Responsibility
Browser to Django	Loads pages, submits password and OTP form values, and carries a session cookie after authentication.
Browser to Firebase	Completes reCAPTCHA and Firebase Phone Auth; receives a signed ID token after provider verification.
Django to Firebase verification	Checks token signature and audience, phone provider, fresh authentication time and expected phone.
Django to Neon PostgreSQL	Reads and writes account, role, school record, challenge and audit state.
Django to email backend	Sends the email challenge after the SMS factor has passed.
GitHub to Vercel	Provides source/version history and build automation when repository integration is configured.

5. The technology, in plain language

Technology	Beginner explanation	What it does here
Python	A programming language used on the server.	Runs business rules and request handling.
Django	A web framework: common building blocks for routing, forms, sessions and permissions.	Implements the portal, authentication flow, school views, audit and admin controls.
PostgreSQL	A relational database that stores structured records in tables.	Stores accounts, challenges, MFA policy and school data.
Neon	A managed cloud PostgreSQL service.	Hosts the documented production database. It is not bundled as a database file on the USB.
Vercel	A cloud platform for deploying web applications and server functions.	Hosts the public portal. A READY deployment does not prove every auth scenario passed live.
Firebase Phone Auth	Google's phone verification service for browser applications.	Runs reCAPTCHA-protected SMS verification and issues a signed ID token after verification.
SMTP / Django email	A mail transport used by the server to send messages.	Delivers email OTP and account messages. The specific server config is private.
GitHub	A hosted Git repository and collaboration service.	Stores versioned source and can trigger build workflows.
Keycloak	An identity and access management server supporting OIDC and other flows.	Optional integration. It is not the Firebase SMS sender; TOTP is typically an authenticator-app code.
reCAPTCHA	A bot-abuse check used in the browser flow.	Must complete before Firebase accepts a phone verification request.

Important separation

Firebase SMS, portal email OTP and Keycloak authenticator-app TOTP are different mechanisms. Configuring one does not automatically configure the others. The current SRS-oriented flow is Firebase SMS first and email OTP second; Keycloak is optional and deployment-dependent.

6. Roles, permissions and account approval

Role	Typical portal abilities	Important boundary
Student	See permitted profile, course, assignment, result and attendance information.	A student must not read or edit another student's private record.
Teacher	Work with assigned courses, students, attendance, assignments and results.	Teacher actions are limited by role and assigned class relationships.
Administrator	Review requests, manage accounts, inspect security activity and administer school records.	Administrative endpoints require authorized administrator permissions.

Approval lifecycle

- A Student or Teacher submits an access request with account details.
- The request remains pending. A pending account does not gain approved portal access.
- An authorized Administrator reviews the request and approves or rejects it.
- After approval, normal sign-in still requires the configured password and OTP flow.
- Django checks role and object ownership on protected operations; hiding a button alone is not authorization.

How to explain authentication versus authorization

Authentication answers 'Who are you, and did you prove it?' Authorization answers 'What may this verified account access?' AuthShield 360 performs both checks; passing OTP does not turn a Student into a Teacher or Administrator.

7. Sign-in: SMS first, then email

This order follows SRS Scenario 3 on pages 9-10. All three portal roles use the same required order when both verification providers are configured.

1

Enter email and password

Django validates password, account state, role approval and logout

2

Complete reCAPTCHA and Firebase SMS

The browser requests and verifies a code for the registered phone

3

Submit Firebase ID token

Django checks signature, project audience, phone provider, freshness and phone match

4

Receive email OTP

Django creates a time-limited, hashed email challenge and sends the second code

5

Enter email OTP

A valid, unused code completes the required verification steps

6

Create portal session

Django grants the role-appropriate dashboard and records the outcome

Current production page check

On 1 October 2026 the public /login/ URL returned HTTP 200 and its visible instructions said to verify by SMS first, then enter an email code if both methods are required. We did not submit credentials, request an SMS or perform an authenticated live sign-in for this document.

8. What each sign-in step proves

Stage	What happens	What it does not prove
Password	Django checks the submitted password using its configured password-hashing mechanism and checks account status.	A correct password alone is not enough for the configured MFA flow.
SMS and reCAPTCHA	The browser asks Firebase to send and verify an SMS code after the security check.	The portal timer cannot override Firebase abuse controls, region rules or carrier filtering.
Firebase ID token	Django verifies a signed provider token, its configured project audience, phone provider, freshness and match to the saved account phone.	A typed code or client-side success banner alone is not trusted by the Django server.
Email challenge	Django generates a second code, stores a password hash of it with send/expiry/attempt state, and sends it to the account email.	A message being accepted by an email transport does not guarantee inbox placement or reading.
Session	After required factors pass, Django establishes an authenticated session and redirects to the role dashboard.	Session creation does not grant access outside the account's role and record scope.

Why the ordering matters

The SMS check happens first because the SRS asks for mobile OTP, followed after success by a time-limited email OTP. Email is the next step-up check. The two channel checks are sequential: the email code is not sent until Django accepts proof that the Firebase phone verification succeeded.

9. Registration, reset and recovery

The sign-in process is not the same as account creation or password recovery. The email challenge has a purpose label so a code issued for one action cannot be casually reused for another.

User action	Expected behavior
Student or Teacher signup	Submits an access request; it stays pending until administrative review. Email verification supports the request flow.
Request status	Uses the request-status path and details associated with the original request. This is distinct from entering the portal.
Forgot password	Starts a purpose-specific, expiring email recovery code. Password recovery does not silently bypass login MFA.
Change password	Requires an authenticated account and follows the dedicated password-change route.
Admin account management	Allows permitted actions such as reviewing access, unlocking accounts and managing MFA policy; each action should be authorized and audited.

Support rule

Do not ask a user to send a password, SMS code, email code, Firebase token, or mailbox password to support. Ask for a redacted error message and the time of the failed attempt instead.

10. OTP limits and lockouts

These are project source defaults documented in the USB README and settings. Production environment overrides may differ. Firebase also applies independent abuse checks and can refuse to send even when the portal allows a request.

Control	Source default	Interpretation
SMS challenge lifetime	300 seconds (5 minutes)	An SMS step expires; an expired provider challenge must be restarted.
Email sign-in OTP lifetime	300 seconds (5 minutes)	Email code is time-limited and single-use after success.
Resend cooldown	300 seconds (5 minutes)	Wait between portal code requests for the same account and channel.
Send ceiling	3 sends per account/channel limit window	Portal stops repeated requests after the configured ceiling.
Send lock	30 minutes after reaching the ceiling	The portal's delivery timer pauses further sends.
Wrong email-code attempts	5 attempts per challenge	The challenge is invalidated after repeated incorrect entries.
Failed password attempts	5 within 15 minutes	Triggers a 5-minute account lock by default.
Repeat password lockout	Escalates to 15 then 30 minutes in the repeat-lock period	Repeated behavior receives a longer pause.
Portal session age	900 seconds (15 minutes)	Configured session expiry; logout and server-side session state also matter.

Do not confuse two limits

AuthShield controls whether the portal permits another request. Firebase separately decides whether it will process SMS based on its own quotas, billing, reCAPTCHA, device, IP, carrier and regional abuse controls. A portal reset does not reset Google's restriction, and a successful SMS once does not guarantee every later request will be sent.

11. MFA settings and Keycloak

The administrator MFA page is at Administrator Dashboard > Security > MFA Settings, route `/administrator/security/mfa/`. The interface stores role policy choices and records administrative changes. In the current code, when both Firebase phone auth and email delivery are ready, every role is required to use both channels, SMS first and email second. The form does not permit relaxing that required combination while both providers are available.

Role-policy edits can invalidate pending OTP challenges for that role; active sessions stay active. These settings govern AuthShield SMS and email MFA, not Keycloak TOTP. The separate Keycloak toggle cannot update production until its Admin API service-account credentials are configured.

Keycloak is an optional identity provider, not a synonym for Firebase

Question	Answer
Does Keycloak send this Firebase SMS?	No. Firebase Phone Auth handles the SMS channel in this integration.
What is Keycloak TOTP?	A time-based code from an authenticator app, not an SMS sent by a carrier.
Can Keycloak replace Firebase?	It can provide identity and authentication features, but SMS requires a supported provider integration. The project has an optional OIDC path; the source folder alone does not prove a live Keycloak server is configured.
Can the dashboard change Keycloak TOTP now?	Not yet. Production is missing <code>AUTHSHIELD_KEYCLOAK_ADMIN_CLIENT_ID</code> and <code>AUTHSHIELD_KEYCLOAK_ADMIN_CLIENT_SECRET</code> , so the dashboard cannot update the Keycloak browser flow. After setup, only a top-level OTP execution is supported; nested OTP subflows need separate work.

12. Data, privacy and audit trail

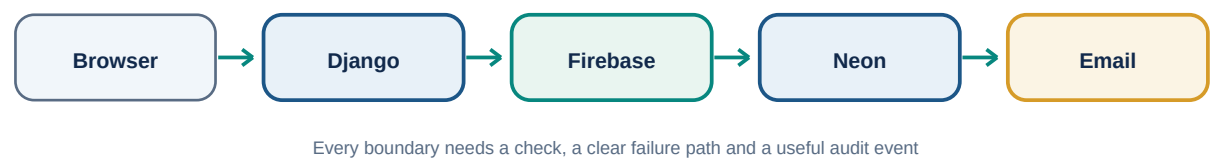
PostgreSQL stores structured data. A simplified account record includes the email, password hash managed through Django authentication, full name, role, phone number, approval state, lockout state and optional external identity subject. Separate tables track email challenges, send limits, MFA policy and audit events.

Data type	Purpose	Handling principle
Password	Prove knowledge of account secret.	Hash it with Django; never put plaintext in logs or PDF.
Phone number	Compare the verified Firebase phone identity with the account.	Treat as personal data; display only what is needed, restrict access and use fictional participant-owned accounts in tests.
OTP challenge	Store purpose, hash, expiry, attempts and completion state.	Keep the readable OTP out of the database and audit trail; expire and consume it.
Firebase ID token	Proof returned after successful provider phone verification.	Verify server-side; do not persist or print raw bearer tokens unnecessarily.
Audit event	Trace a security-relevant decision or administrative action.	Record actor/target, action, factor and outcome; exclude passwords, codes and raw session cookies.

Audit events are useful when the team can answer: who acted, what happened, which factor or control was involved, when it occurred, and whether it succeeded. SRS asks for new authentication events to appear in the monitoring interface within five seconds under normal lab conditions. This review did not measure that live visibility target; collect a timestamped demonstration as evidence.

13. Security controls and realistic limits

Security is a set of layers. Each layer can reduce risk, but none makes a public web application invulnerable. This section describes controls visible in the current project and distinguishes them from external testing that has not been evidenced here.



Threat area	Relevant control in this project	What still needs validation
Stolen password	Password plus sequential SMS and email checks when both providers are configured.	Phishing-resistant MFA, safe recovery, real-world user training and account recovery review.
Brute-force login	Account-level failed-login limits, escalating lockouts and OTP attempt limits.	Verify exact live overrides, IP behavior, account enumeration and distributed abuse in authorized tests.
OTP spam/cost	Per-account/channel cooldown, send ceiling and lock period.	Inspect Firebase quotas, abuse/region controls and actual billing alerts separately.
Forged SMS success	Django verifies Firebase-signed ID token claims and expected phone.	Test invalid, expired, wrong-project and replayed tokens in a controlled environment.
Cross-role access	Django role checks and ownership/assignment boundaries.	Run the required Student-to-Teacher/Admin and Teacher-to-Admin access matrix.
Session theft/reuse	Secure-cookie settings and 15-minute default session age are configured.	Test cookie flags, logout invalidation, session rotation, timeout and old-session reuse.
Injection or script execution	Django forms/templates and ORM provide framework protections when used correctly.	Perform code review and authorized DAST; framework choice alone does not prove absence of XSS or SQL injection.

Security claim

No scan can prove that a system is impossible to hack. This guide does not claim a full OWASP ZAP/Burp assessment, formal penetration test, complete accessibility audit, or vulnerability-free status. Test only the authorized AuthShield environment and fictional or participant-owned accounts.

14. SRS traceability: implementation versus evidence

The SRS describes both application behavior and submission evidence. Code may implement a control while the team still needs a screenshot, log, measured result or video to prove it for evaluation.

SRS item	Current source / deployment status	Evidence status for handoff
Fictional school portal and three roles	Portal routes and role views exist; deployment is live.	Owner/team reports OTP worked; collect a fresh role-by-role demo record.
Scenario 3: mobile OTP then email OTP	Implemented SMS first, then email; live login page copy verified 1 Oct 2026.	No authenticated live test was performed for this guide; preserve the team's own safe evidence.
Valid, invalid, expired OTP and no-MFA denial	Automated tests exercise important OTP policy and verification paths.	SRS evidence screenshots/logs for each case are not in this PDF or USB evidence pack.
Failed-login protection	Project defaults include 5 failures in 15 minutes and escalating lockouts.	Record the exact test, lockout time, recovery/reset and corresponding audit event.
Role access control	Django applies role-based protected views and account-scoped data paths.	Required cross-role negative tests and screenshots still need a dedicated matrix.
Logs visible within 5 seconds	Audit recording and admin security log exist in source.	The SRS five-second display target has not been measured here.
Average login time, 3 runs	Local isolated test runs measured three runs per role and mode.	These are server-side mocked-provider timings, not live browser/carrier timings.
Controlled restart and persistence	PostgreSQL-backed state is used; source tests cover persistence behavior.	Attach a timestamped restart/reload demonstration and confirm records/settings remain.
Authorized Kali/ZAP/Burp testing	The project supports a local/authorized test environment.	No completed DAST/penetration-test report is included here.
Presentation and mandatory demo video	Not verified by code or deployment status.	Still produce the requested presentation/video and include required scenarios.
README and consolidated package	One README and the USB project folder are present.	Ensure the final team ZIP contains source, README and approved evidence without secrets.

Accurate project status

The deployed application implementation is in place and its automated suite passed. The full SRS deliverable set is not proven complete until the team fills the evidence matrix, demonstrates the five-second event target, records controlled restart and role tests, and submits the required video and report evidence.

15. Test results and what they mean

On 1 October 2026 the full Django suite was run with isolated test settings. Result: 136 tests passed in 20.357 seconds, and Django reported no system-check issues. A focused OTP-order suite also passed 20 tests in 1.862 seconds.

What automated tests establish

- The tested code paths behaved as expected with the test database and test settings.
- OTP ordering, permissions, throttling and account workflow logic have automated checks.
- Mocked Firebase/email provider tests help validate portal logic without spending money or sending real messages.

What they do not establish

- That Firebase accepted a live send request or that a particular carrier delivered a message.
- That the real production environment has the same secret values or overrides as the isolated test settings.
- That every browser, mobile device, role, cloud region, attack technique or deployment route is safe.

Check	Result / meaning
Full Django suite	136 passed, 20.357 seconds; isolated test settings.
Focused OTP flow suite	20 passed, 1.862 seconds.
Django system check	No issues reported in that test run.
Live public login route	HTTP 200; rendered page states SMS first and email second when both required.
Live OTP send/login	Not performed by Codex during this guide's preparation.

16. Performance measurements: read the caveat

The project includes a repeatable local performance test that runs each test scenario three times for Student, Teacher and Administrator fixtures. The numbers below came from an isolated Django test database and local email backend. Firebase token verification was mocked in the SMS-first scenario. These timings measure server-side test requests, not a real user waiting for a mobile network, email inbox or production cloud region.

Scenario	Student mean	Teacher mean	Admin mean	What is included
Password-only baseline	15.70 ms	15.31 ms	14.17 ms	Local password and session path.
Password + email OTP	15.18 ms	14.72 ms	13.73 ms	Local email backend; no carrier delay.
Password + SMS then email	34.15 ms	31.44 ms	26.12 ms	Local Django path; Firebase proof mocked.

How to present these numbers

Say: 'These are three-run local server-side measurements. SMS provider and network wait time are not included, so we will separately time the full browser flow on an owned test account for realistic user experience.' Do not describe the local mean as live OTP delivery time.

For a jury-ready measurement, record start/end timestamps, account role, browser/device, network conditions, provider result, success/failure and all three run durations for each SRS scenario. Report the mean and keep the raw observations.

17. Source map and project handoff

Path	What a maintainer finds there
accounts/	User model, password login, MFA policy, OTP challenge and limits, Firebase token checks, recovery, Keycloak callback and tests.
school/	Student, Teacher and Administrator screens, school records, approvals, attendance and management operations.
config/	Django settings, routes, test settings and application entry points.
templates/	HTML pages for sign-in, OTP, email, dashboard and school workflows.
static/	Styles and browser JavaScript, including Firebase phone verification and OTP input behavior.
keycloak-vercel/	Optional Keycloak-related files; presence alone does not mean a service is deployed.
.github/	Repository workflow configuration.
README.md	Single project setup and handoff guide; never add credentials to public docs.

Cloud ownership is separate from the source folder

The USB carries project files, not ownership of GitHub, Vercel, Neon, Firebase, Google email or an optional Keycloak service. A new authorized maintainer needs access granted to each service. Do not hand out a workbook or .env file casually: the USB copy may include private local configuration material. The PDF deliberately omits passwords, OTPs, API key values, connection strings and personal phone numbers.

Repository: github.com/mustafaahmed-sec/AuthShield360

Portal: authshield360.vercel.app

18. Deployment and safe operation

Production deployment uses Vercel for the portal and Neon PostgreSQL for hosted data. In the verified deployment on 1 October 2026, Vercel marked the production build READY and attached the public alias <https://authshield360.vercel.app>. The public login route returned HTTP 200 after deployment. This confirms availability of that page, not an end-to-end OTP login.

A safe deployment checklist

- Use separate production and test credentials; keep secret values in the hosting provider's secure environment settings.
- Confirm the production database connection and run reviewed migrations before routing users to a new version.
- Confirm Firebase authorized domains, project identifiers, billing, phone region policy, reCAPTCHA and App Check settings as applicable.
- Check email sender settings and perform a controlled delivery test with a team-owned inbox.
- Verify role access and all failure paths after deploy using approved test accounts.
- Check build logs and deployment health without copying tokens or private configuration into a report.

Dated Vercel usage observation

A screenshot supplied by the team on 30 September 2026 showed Functions Storage at 11.18 GB / 10 GB at team level. That is a historical screenshot, not a fresh usage measurement. Recheck Vercel Usage before concluding whether the limit is still exceeded or which project contributes most.

19. Safe demonstration plan

Use only fictional accounts or accounts owned by the participants, and only the authorized AuthShield environment. Coordinate the test SMS and mailbox before recording. Never show real passwords, bearer tokens, email app passwords, database URLs, OTPs that remain valid, or private account data in a public video.

Demo moment	Show	Capture as evidence
1. Landing and roles	Portal purpose and role entry points.	Portal URL and visible date, no private user data.
2. Pending request	Submit a fictional Student/Teacher request and show pending state.	Request ID or redacted status and admin review.
3. Password failure	Use a deliberately incorrect test password.	Denied result and relevant audit event.
4. SMS first	Complete reCAPTCHA, receive Firebase SMS on participant-owned phone, enter code.	Redacted page and successful SMS factor event; hide the code itself.
5. Email second	Retrieve the purpose-labeled email and enter the email code.	Redacted success state and audit event; hide the valid code.
6. Role restriction	Attempt a prohibited resource with a low-privilege fictional role.	Access denied and audit record, if available.
7. Lockout and recovery	Use controlled failed attempts, show timer and authorized reset path.	Start/end times and relevant admin action.
8. Logout and session	Log out, then attempt to reuse the prior browser session.	Redirect/denial and logged outcome.

Record enough browser and monitor view to show that new authentication events appear within the SRS target of five seconds. Keep the evidence in the team's approved private submission location.

20. Troubleshooting guide

Symptom	Check in this order
SMS code does not arrive	reCAPTCHA completion; configured Firebase project; authorized domain; international phone format; billing; allowed destination regions; Firebase throttling; carrier delivery.
Firebase shows an internal or restricted-carrier error	Record exact code and timestamp. Check Firebase project usage/Phone Auth controls. Wait for provider policy reset or contact Firebase support; portal reset cannot clear a provider restriction.
SMS succeeds but no email field appears	The email step should follow successful Firebase token verification. Restart the flow if challenge expired; check the exact deployed revision and browser page.
Email arrives but code is rejected	Use the latest code for the current sign-in, check expiry/copy spaces, purpose and resend status, and avoid older messages.
Valid password is rejected	Check account approval, account active state, role, lockout timer and whether the person used login rather than request-status.
New code button is disabled	Read the timer and send limit. Wait for cooldown or ask an authorized admin to reset the portal lockout if allowed.
Keycloak page or error appears	Check whether optional Keycloak login is enabled and the configured service/realm is reachable. Keycloak is separate from Firebase SMS.
Works locally but not on Vercel	Compare variable names/config privately, migration state, authorized domains, deployment logs and service status. Never paste secret values in a chat or report.

21. Jury questions and concise answers

Question	Answer the team can explain
What problem does the project address?	It demonstrates why school portal accounts should use layered verification and role checks instead of relying on a password alone.
Why SMS before email?	SRS Scenario 3 describes mobile OTP first and then time-limited email OTP. We changed the portal to follow that order.
Does the browser decide whether SMS succeeded?	No. Django validates Firebase's signed ID token, expected project, phone provider, freshness and account-phone match.
What is the difference between authentication and authorization?	Authentication proves identity; authorization decides which resources that identity may use.
Why have both Firebase and Keycloak?	They serve different integrations. Firebase handles the current SMS step. Keycloak is an optional identity provider and can support authenticator-app TOTP; it does not automatically send SMS.
Can rate limits stop every SMS charge?	They reduce repeated portal requests, but Firebase also has independent abuse checks and billing. They cannot guarantee delivery or eliminate all cost.
What happens after three OTP sends?	Source defaults apply a 30-minute portal send lock after the configured ceiling, with a five-minute cooldown. Live environment overrides and Firebase limits must be checked.
Is the project unhackable?	No. Security is risk reduction. The team must keep testing, patching, monitoring and protecting service credentials.
Did 136 tests prove live SMS?	No. They passed under isolated test settings; provider behavior was simulated. The owner/team separately reported successful OTP use.
Is the project fully complete to the SRS?	The deployed implementation is present, but SRS evidence items such as a complete test matrix, measured five-second audit visibility, authorized DAST evidence and mandatory demo video still need confirmation.
What is the most important next deliverable?	Capture a repeatable evidence matrix and demo video using fictional or participant-owned accounts in the authorized environment.

22. Roadmap: the next professional improvements

Priority	Improvement	Why it matters
P0	Finish the SRS test matrix with actual result, role, timestamp, screenshot/log reference and pass/fail for every required case.	This turns implemented behavior into auditable submission evidence.
P0	Record the mandatory demo video and show SMS-first, email-second, invalid/expired OTP, lockout, role denial, audit and logout behavior.	The SRS explicitly asks for a demonstration video.
P0	Measure authentication-event visibility against the five-second SRS target under lab conditions.	The code's existence does not prove the visible timing requirement.
P1	Perform an authorized code review and DAST with OWASP ZAP or Burp in an isolated target environment.	Finds issues that unit tests cannot, including browser, cookie and access-control behavior.
P1	Add provider health and delivery observability without logging OTPs or full phone numbers.	Helps distinguish portal throttles, Firebase restrictions, SMTP errors and carrier delays.
P1	Document recovery and contact-change procedures with strong reauthentication and audit.	A secure sign-in flow must be matched by secure recovery and MFA reset.
P2	Evaluate phishing-resistant WebAuthn/passkeys for privileged accounts.	SMS and email OTP can be phished or socially engineered.
P2	Add backup/recovery codes or a managed authenticator option with enrollment verification.	Improves availability when a phone is lost or roaming.
P2	Review Vercel storage/usage and clean obsolete deployments after identifying owners and rollback needs.	Reduces hosting pressure without deleting active rollback assets blindly.

A maintainable definition of done

A professional release has a traceable change, automated checks, a ready deployment, a verified live path, a rollback plan, a documented test result and a clear owner. Avoid calling the product complete until each SRS deliverable has either evidence or a documented approved exception.

23. Glossary for a new team member

Term	Plain-language definition
Authentication	Checking that a person controls the identity they claim.
Authorization	Checking whether an authenticated account may perform a requested action.
MFA	Multi-factor authentication. More than one independent category of proof is used.
OTP	One-time password/code, which expires and is accepted once for a particular challenge.
SMS	Text message delivered through a mobile carrier.
TOTP	Time-based one-time password from an authenticator app, based on a shared secret and clock.
ID token	A signed statement from an identity provider that a client can present for server verification.
reCAPTCHA	A challenge or risk check that helps reduce automated abuse.
Session	Server-managed state that remembers an authenticated browser between requests.
Audit event	A structured record that explains a security-relevant action and its outcome.
Rate limit	A rule that caps how many requests can happen in a period.
Neon	A hosted PostgreSQL database service.
Vercel deployment	A built and hosted version of the web application.
OIDC	OpenID Connect, an identity layer commonly used for sign-in with an external provider.
DAST	Dynamic application security testing against a running, authorized application.

24. Sources and reference links

Project sources reviewed

- AuthShield360 README.md in the USB project folder, read on 1 October 2026.
- AuthShield360 source: accounts/mfa.py, accounts/otp.py, accounts/views.py, accounts/models.py, accounts/tests.py, accounts/test_performance.py, config/settings.py, templates/accounts/login.html and templates/accounts/otp_verify.html.
- AuthShield 360 - Ethical Cyber Horizons SRS, 21 pages. Scenario 3: pages 9-10; security and testing expectations: pages 14-17; AI-use note: page 18; deliverables: pages 20-21.
- Production deployment command result and public /login/ HTTP response checked 1 October 2026. No live sign-in was performed for this PDF.

Official technical references

- Django authentication
- Django sessions
- Firebase web phone authentication
- Firebase Admin SDK token verification
- Neon: what is a serverless database?
- Vercel Python runtime
- GitHub: About Git
- Keycloak Server Administration Guide
- OWASP Authentication Cheat Sheet
- OWASP Multifactor Authentication Cheat Sheet

Final summary

AuthShield 360 is a Django school portal deployed on Vercel with a documented Neon PostgreSQL database, Firebase Phone Auth for SMS, and server-sent email OTP. The SRS-oriented sign-in order is now password, SMS, then email. Automated tests pass, and the live login page reflects that order. The remaining work is evidence: demonstrate and measure the complete flow, finish the SRS security matrix, verify the five-second audit target, and deliver the mandatory video. The project should be described as deployed and implemented, with those evidence items still to be completed or confirmed.